IBM Innovation Centre for Education

NCS Nihon Communication Solutions

Yenepoya Institute of Arts, Science, Commerce, and Management

Final Project Report

on

Cloud Forensics Automation for Retail Breaches

Student Name: Saibin K Siju

Industry Mentor: Mr. Shashank Korram **Guided by: Ms. Prathiksha Mudipu**

Table of Contents

Section No.	Title	Page No.
1	Executive Summary	4
2	Background	5
2.1	Aim	5
2.2	Technologies	6
2.3	Hardware Architecture	9
2.4	Software Architecture	11
3	System	13
3.1	Requirements	13
3.1.1	Functional Requirements	13
3.1.2	User Requirements	14
3.1.3	Environmental Requirements	15
3.2	Design and Architecture	15
3.3	Implementation	20
3.4	Testing	22
3.5	Graphical User Interface Layout	25

3.6	Customer Testing	29
3.7	Evaluation	30
4	Snapshots of the Project	33
5	Conclusion	41
6	Further Developement On Research	43
7	References	46
8	Appendix	48-50

1. EXECUTIVE SUMMARY

The rapid growth of digital systems in retail environments has significantly increased the risk of cyber threats such as unauthorized file modifications, data breaches, and tampering of sensitive information. Manual detection of such incidents is inefficient, highlighting the need for automated forensic solutions.

This project, Cloud Forensics Automation for Retail Breaches, presents an automated system for monitoring, detecting, and analyzing file-level changes. The system continuously monitors a designated directory and identifies file additions, deletions, and modifications using cryptographic hashing techniques.

The core functionality is based on SHA256 hashing, ensuring accurate detection of even minor file changes. Each file is classified as NEW, MODIFIED, UNCHANGED, or DELETED, and assigned a corresponding risk level. All forensic evidence is securely stored in a PostgreSQL database, ensuring data integrity and traceability.

The system is implemented using a Flask-based web application, providing a client interface for file operations and an admin dashboard for forensic analysis. The dashboard offers real-time insights through charts, logs, and summaries, enabling quick identification of potential threats.

A real-time monitoring mechanism using a watchdog module ensures automatic detection of file changes without manual intervention. Additionally, the system supports CSV export functionality for easy reporting and external analysis.

Overall, the project provides an efficient, scalable, and user-friendly solution for automated forensic monitoring, enhancing security and enabling rapid detection of retail data breaches.

2. BACKGROUND

In modern retail environments, digital systems play a critical role in managing operations such as billing, inventory management, customer records, and transaction processing. These systems store large volumes of sensitive data, making them prime targets for cyberattacks and unauthorized access. One of the major challenges in such environments is detecting unauthorized modifications to files. Attackers may alter transaction records, manipulate pricing data, or delete logs to hide their activities. Traditional security systems often focus on network-level threats but fail to detect subtle file-level changes effectively.

Digital forensics has emerged as a crucial field in Cybersecurity, focusing on the identification, preservation, and analysis of digital evidence. However, manual forensic analysis is time-consuming and requires specialized expertise. There is a growing need for automated solutions that can continuously monitor systems and provide real-time insights into potential security breaches.

This project addresses this gap by introducing an automated cloud-based forensic system capable of detecting file-level changes using cryptographic techniques. By leveraging technologies such as Python, Flask, PostgreSQL, and real-time file monitoring, the system provides a scalable and efficient solution for retail breach detection.The system simulates a real-world retail breach scenario, where files may be modified or deleted without authorization. By automatically detecting and classifying these changes, the system helps forensic investigators identify suspicious activities quickly and accurately.

2.1 Aim

The primary aim of this project is to design and develop an automated forensic system that monitors file-level activities in a retail environment and detects unauthorized changes in real time.

The objectives of the project include:

- To develop a system that continuously monitors a directory for file changes
- To implement SHA256 hashing for accurate tamper detection
- To classify files into categories such as NEW, MODIFIED, UNCHANGED, and DELETED

- To assign risk levels based on the severity of detected changes
- To store forensic evidence securely in a PostgreSQL database
- To provide a web-based dashboard for visualization and analysis of forensic data
- To enable real-time monitoring using file system event detection
- To allow exporting of forensic reports for further investigation

The project aims to enhance digital security by providing an efficient and automated solution for detecting potential data breaches in retail systems.

2.2 Technologies

The system is developed using a combination of modern web technologies and forensic tools to ensure performance, scalability, and reliability.

- **Backend**

Python: Python serves as the core programming language for the system. It is used to implement the forensic logic, file handling operations, and backend processing. Python's simplicity and extensive library support make it ideal for developing security-focused applications.

Flask Framework: It is used as the web framework to build the backend application. It handles routing, API development, request processing, and integration with other system components. Flask provides a lightweight and flexible environment for building scalable web applications.

- **Frontend**

HTML5: It is used to structure the web pages of the system, including the client interface and admin dashboard. It organizes content using semantic elements, improving readability and maintainability.

In this project, HTML5 is used for:

- Designing user interfaces and dashboards
- Creating forms for file uploads and inputs
- Displaying forensic data in structured layouts

CSS3: CSS is used to style the application and enhance its visual appearance. It ensures a clean, responsive, and consistent layout across devices.

It is used for:

- Designing responsive layouts
- Styling dashboards, tables, and buttons
- Improving user experience with proper spacing and visuals

JavaScript: JavaScript adds dynamic functionality to the application. It enables interaction with backend APIs and updates the interface without reloading pages.

It is used for:

- Fetching and displaying data from APIs
- Handling user interactions and form submissions
- Updating dashboard data in real time

Chart.js: It is used for data visualization in the dashboard. It presents forensic data using charts such as bar, pie, and line graphs.

It helps to:

- Visualize file status and risk levels
- Display trends and system activity
- Improve data understanding through graphical representation

- **Report Export Technology**

CSV (Comma-Separated Values): CSV is used as a report export format in the system to generate and download forensic evidence reports. The system dynamically creates CSV files from the PostgreSQL database whenever the user requests an export.Unlike traditional systems where CSV is used for data storage, this project uses CSV only for reporting purposes, ensuring that the primary data remains secure and consistent within the PostgreSQL database.

The CSV export feature allows users to:

- Download the latest forensic scan results
- Share reports easily with investigators or stakeholders
- Perform offline analysis using spreadsheet tools like Microsoft Excel

The CSV file is generated in real time using Python's built-in csv module, without storing the file permanently on disk. This approach improves security and reduces storage overhead

- **Database**

PostgreSQL: PostgreSQL is used as the primary relational database for storing forensic evidence. It provides strong data integrity, reliability, and efficient query processing. All scan results, including file hashes, status, and timestamps, are stored in a structured format, enabling easy retrieval, analysis, and reporting.It supports advanced features such as ACID compliance, indexing, and relational data management, making it suitable for secure and scalable applications.

Psycopg2: It is a Python library used to connect the application with the PostgreSQL database. It enables execution of SQL queries, data insertion, and retrieval operations.It also supports transaction management, ensuring that database operations are performed securely and consistently within the system.

- **Real-Time Monitoring**

Watchdog Library: This is used to monitor file system events in real time. It continuously observes the designated directory and detects changes such as file creation, modification, and deletion. When a change is detected, the system automatically triggers the scanning process, ensuring immediate analysis and detection of any unauthorized modifications. This enables real-time forensic monitoring without manual intervention. The use of Watchdog improves system responsiveness and supports quick identification of potential security threats.

- **Security**

Werkzeug : The `secure_filename()` function from the Werkzeug library is used to sanitize file names before storing them in the system. It removes unsafe characters and ensures that file paths remain valid and secure. This helps prevent path traversal attacks and unauthorized file access by restricting malicious file names, thereby enhancing the overall security of file handling within the application.

Subprocess Module: It is used to execute the scanning engine as a separate process. This ensures that the main application remains isolated from the scanning logic. By running the scanning operation independently, the system improves reliability and reduces the risk of crashes or security issues affecting the core application. It also provides better control over execution and error handling.

2.3 HARDWARE ARCHITECTURE

The system follows a distributed hardware architecture to ensure reliability, performance, and scalability. It is designed to operate across multiple logical components, including client devices, application server, database server, and file monitoring system. The client side consists of user devices such as desktops, laptops, or mobile devices, which access the system through a web browser. These devices interact with the application and send requests over the internet using secure protocols. The application server hosts the Flask-based backend, which processes incoming requests, executes business logic, handles file operations, and communicates with other system components.

The database server runs PostgreSQL, which stores all structured forensic data, including file hashes, scan results, and logs. It ensures secure, consistent, and efficient data storage and retrieval. The file monitoring system continuously observes the scan folder using the Watchdog library. It detects file creation, modification, and deletion, and triggers the forensic scanning process in real time. All components communicate over HTTP/HTTPS protocols, ensuring secure and efficient data exchange. Security measures such as controlled access, input validation, and secure communication help protect system resources.The architecture supports scalability by allowing components to be managed independently. Backup mechanisms and logging systems can be used to ensure data reliability and track system activity.

Overall, this distributed architecture improves fault tolerance, enhances system performance, and enables efficient coordination between all components.

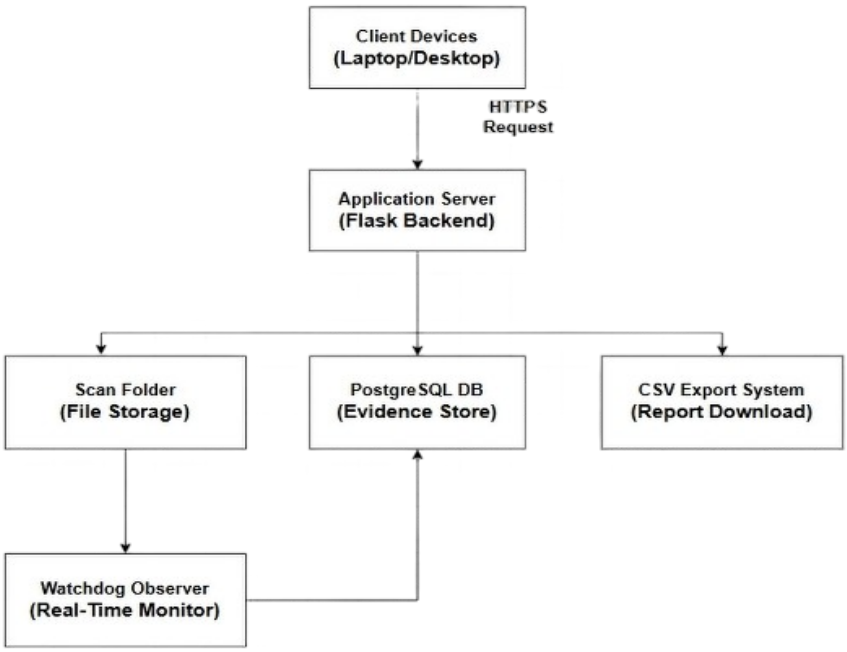


Figure 1.3: Hardware Architecture Diagram

2.3.1 Network Communication Flow

The network communication flow defines how different components of the system interact securely and efficiently. Client devices communicate with the application server using HTTPS protocols, ensuring encrypted transmission of data. The application server processes incoming requests and interacts with the PostgreSQL database using SQL queries to store and retrieve forensic data. The file monitoring system communicates internally with the application to trigger scanning operations. Secure communication mechanisms, request validation, and controlled data exchange ensure reliable interaction between all system components.

2.3.2 Security at Hardware Level

Security is implemented at the hardware and infrastructure level to protect system components from unauthorized access and attacks. Secure communication is enforced using HTTPS encryption between client devices and the application server. Access control mechanisms restrict unauthorized access to system resources such as the database and application server. Input validation and secure configurations help prevent malicious activities. Logging and monitoring mechanisms can be used to track system activity and detect potential security threats. These measures enhance system reliability and protect sensitive forensic data.

2.4 SOFTWARE ARCHITECTURE

The system follows a layered software architecture to ensure modularity, scalability, and maintainability. It is designed by separating the system into presentation, application, and data layers, allowing clear distribution of responsibilities and efficient communication between components. The Presentation Layer handles user interaction and provides the interface through which users access the system. It is developed using HTML5, CSS3, and JavaScript, enabling responsive design and dynamic content updates. This layer communicates with the backend through REST API calls. The Application Layer is implemented using Python Flask and acts as the core processing unit of the system. It manages API routing, business logic, file processing, and forensic scanning operations. It also handles data validation, SHA256 hash

computation, and risk classification. This layer ensures secure processing of requests and coordination between system components. The Data Layer uses PostgreSQL to store and manage structured forensic data, including file hashes, scan results, and logs. It ensures data integrity, secure storage, and efficient retrieval of information.

This layered architecture improves maintainability by separating responsibilities, supports scalability, and enables efficient communication between all modules. It also simplifies future system enhancements and integration of additional features.

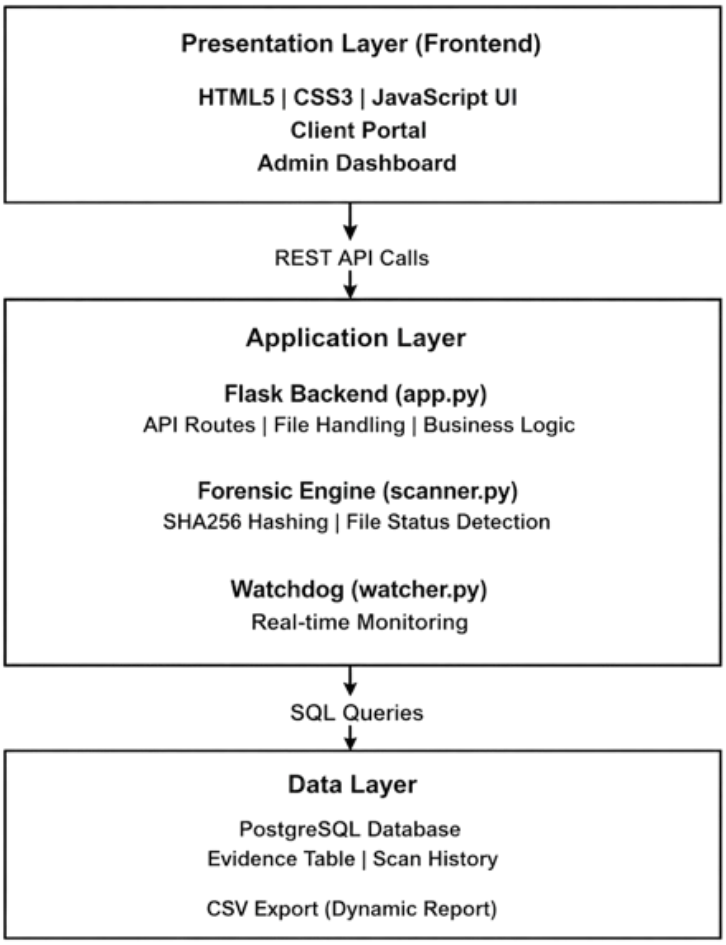


Figure 1.4: Software Architecture Diagram showing Presentation, Application, and Data layers with REST API and database interaction

3. SYSTEM

The Cloud Forensics Automation for Retail Breaches system is a web-based application designed to monitor, detect, and analyze file-level changes within a retail environment. The system focuses on identifying unauthorized modifications, deletions, or additions of files using cryptographic techniques and automated scanning mechanisms. The system operates by continuously monitoring a designated directory (scan_folder) and performing forensic analysis whenever changes are detected. It uses SHA256 hashing to generate unique digital fingerprints for each file, enabling accurate detection of even minor modifications.

The application provides two main interfaces:

- A client interface for file upload and management
- An admin dashboard for forensic monitoring and analysis

The system integrates real-time monitoring, secure database storage, and data visualization tools to provide a complete and efficient forensic solution.

3.1 REQUIREMENTS

System requirements define the essential functionalities and conditions necessary for the successful operation of the application.

3.1.1 Functional Requirements

Functional requirements describe what the system is expected to perform.

The system must:

- Allow users to upload files through a web interface
- Store uploaded files securely in the *scan_folder* directory
- Automatically detect file creation, modification, and deletion
- Generate SHA256 hashes for all files
- Compare current file hashes with previous records
- Classify files into:
    - NEW
    - MODIFIED

    - UNCHANGED
    - DELETED
- Assign risk levels:
    - CRITICAL
    - HIGH
    - MEDIUM
    - LOW
- Store forensic evidence in a PostgreSQL database
- Provide an admin dashboard displaying:
    - File status
    - Risk levels
    - Scan history
- Support real-time updates using automatic refresh
- Allow users to view, edit, and download files
- Provide CSV export of forensic reports
- Allow manual scan triggering through API

3.1.2 User Requirements

User requirements define the expectations and usability needs of system users.

The system should:

- Provide a simple and user-friendly interface
- Allow easy file upload and management
- Display clear and understandable forensic reports
- Show color-coded status indicators for quick analysis
- Enable users to export reports for offline use
- Provide fast response and real-time updates
- Allow smooth navigation between dashboard and file operations

3.1.3 Environmental Requirements

Environmental requirements define the system's operational conditions and setup.

The system requires:

- A web browser (Chrome, Edge, Firefox)
- Python 3.8 or higher
- PostgreSQL database server
- Operating system (Windows/Linux/macOS)
- Stable network connection

Minimum Hardware Requirements:

- 4 GB RAM
- Dual-core processor

3.2 DESIGN AND ARCHITECTURE

The system design follows a modular and layered approach to improve maintainability, scalability, and performance. It is based on the principle of separation of concerns, where each module performs a specific function while interacting with other components through well-defined interfaces. The system is divided into functional modules such as file handling, forensic scanning, database management, and real-time monitoring. Each module is designed independently to simplify development, testing, and future enhancements.

The architecture follows a RESTful API-based design, enabling communication between the frontend and backend. This ensures loose coupling between components and allows easy integration of additional features in the future.

The design also incorporates security considerations such as input validation, secure file handling, and controlled access to system resources. These measures help protect the system from unauthorized access and ensure safe data processing.

The architecture supports scalability and extensibility, allowing future improvements such as cloud deployment, advanced analytics, and enhanced monitoring capabilities.

3.2.1 High-Level Design Overview

The High-Level Design (HLD) defines the overall structure of the system and the interaction between major components.

The system follows a two-tier architecture consisting of:

- Frontend (client interface)
- Backend (application logic and database)

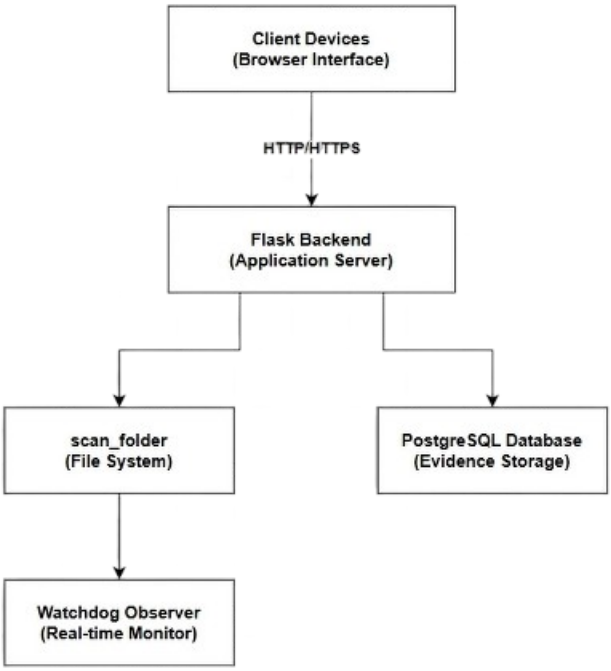


Figure 2.2.1: High-Level Architecture Diagram

The diagram illustrates how client devices interact with the Flask backend through HTTP/HTTPS requests. The backend processes requests, performs forensic analysis, and communicates with the PostgreSQL database. The file monitoring component ensures real-time detection of file changes.

3.2.2 Process Flow Design

The system operates through two main flows:

1. File Upload Flow

- User uploads file via client interface
- File is stored in *scan_folder*
- Scanner is triggered
- SHA256 hash is generated and stored

2. Real-Time Monitoring Flow

- Watchdog detects file change
- Scanner is triggered automatically
- File status is updated

This diagram shows the step-by-step process from file upload to forensic analysis and dashboard display:

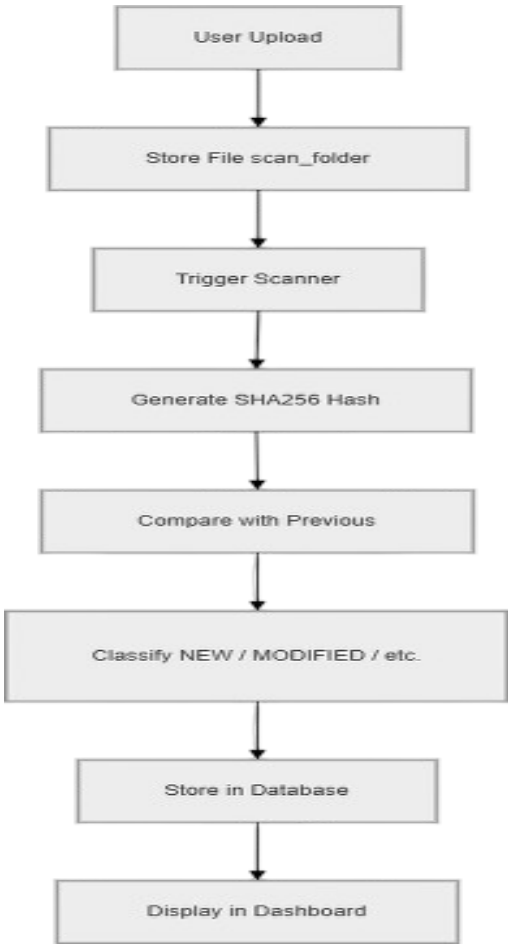


Figure 2.2.2: Process Flow Diagram illustrating file upload and forensic scanning workflow.

3.2.3 Data Flow Design

The system processes data through multiple stages:

- Input: File upload or file system event
- Processing: Hash generation and comparison
- Storage: PostgreSQL database
- Output: Dashboard display and CSV export

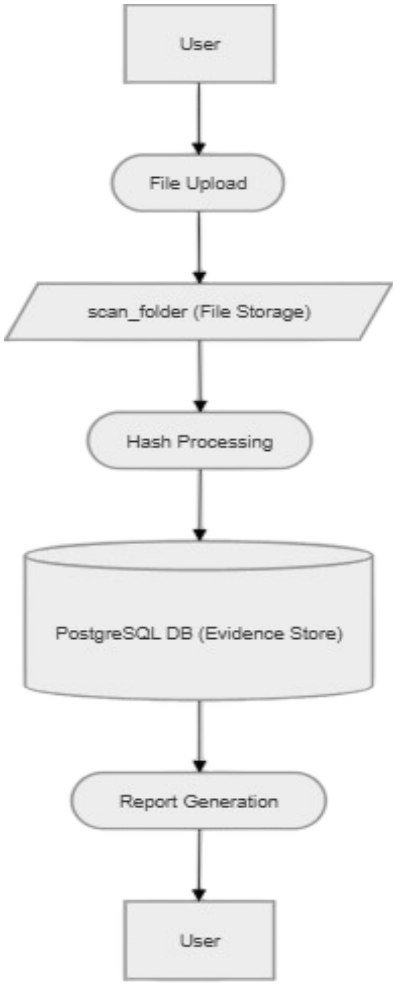


Figure 2.2.3: Data Flow Diagram (DFD Level 1)

This diagram illustrates the flow of data in the system, starting from user file upload, storage in the scan folder, hash processing, evidence storage in PostgreSQL, and finally report generation for the user.

3.2.4 Component Design

The system is divided into the following components:

1. **Flask Application (app.py):** Handles routing, APIs, and request processing
2. **Forensic Engine (scanner.py):**Computes SHA256 hashes and detects file changes
3. **Database Layer (db.py):**Stores and retrieves forensic evidence
4. **File Monitoring (watcher.py)**:Detects real-time file system changes
5. **Frontend Interface**: Provides user interaction and dashboard

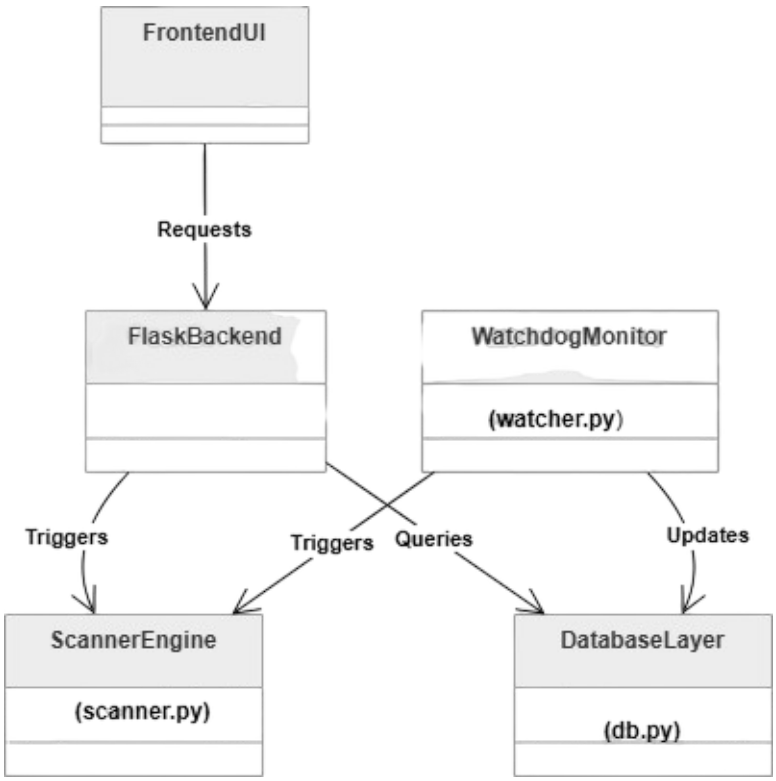


Figure 2.2.4: Component Diagram

This diagram illustrates how the major components of the system interact. The frontend communicates with the Flask backend, which coordinates the scanner engine and database layer. The watchdog monitor continuously observes file changes and triggers the scanning process.

3.2.5 API Design

The system provides RESTful APIs for communication:

- `/api/evidence` → Fetch latest scan results
- `/api/summary` → Get dashboard statistics
- `/api/history` → Retrieve scan history
- `/api/export` → Download CSV report
- `/api/scan` → Trigger manual scan

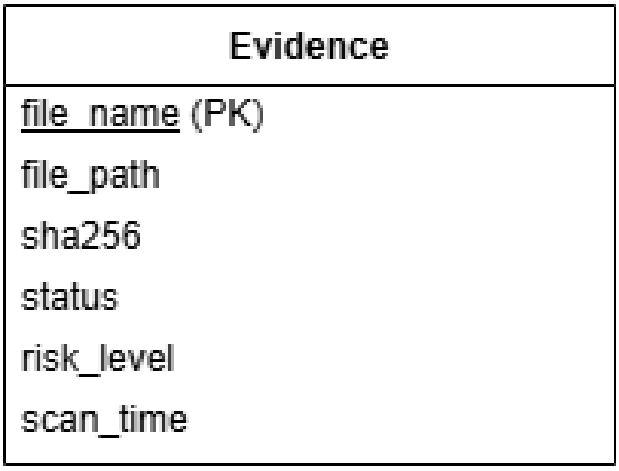


Figure 2.2.5: ER Diagram

3.3 Implementation

The system is implemented using a modular full-stack approach that integrates backend processing, file monitoring, database management, and frontend interaction. The implementation ensures efficient performance, scalability, and secure handling of forensic data. Each module is designed to operate independently while maintaining seamless communication with other components.

3.3.1 Backend Implementation: The backend is developed using the Flask framework in Python. It acts as the central controller of the system, handling API requests, processing business logic, and coordinating between different modules. It performs operations such as routing, file handling, triggering forensic scans, and retrieving data from the database. The backend follows a RESTful architecture, enabling efficient communication and structured request-response handling between system components.

3.3.2 File Handling Implementation: The system supports secure file upload and storage in a designated directory called *scan_folder*. Files are validated and sanitized using secure methods such as *secure_filename()* to prevent security vulnerabilities like path traversal attacks.

The module also handles file retrieval, updates, and deletion while maintaining data integrity.

3.3.3 Forensic Engine Implementation: The forensic engine is responsible for detecting file changes. It uses SHA256 hashing to generate unique file signatures and compares them with previously stored hashes.

Based on comparison, files are classified as:

- NEW
- MODIFIED
- UNCHANGED
- DELETED

This mechanism ensures accurate detection of even minor changes in file content.

3.3.4 Database Implementation: The system uses PostgreSQL to store forensic evidence, including file metadata, hash values, status, and timestamps. Secure database operations are ensured using parameterized queries to prevent SQL injection attacks. The structured schema design supports efficient querying and data retrieval.

3.3.5 Real-Time Monitoring Implementation: The system uses the watchdog library to monitor file system events such as file creation, modification, and deletion in real time.

Whenever a change is detected, the watchdog automatically triggers the forensic scanning process, ensuring continuous monitoring and immediate detection of file modifications.

3.3.6 Additional Features: The system includes CSV report generation for exporting forensic data, allowing easy sharing and offline analysis. The frontend interface provides visualization and interaction capabilities for users to view results and manage files.

3.3.7 System Integration: All modules are integrated to function as a unified system. The frontend communicates with the backend through REST APIs, while the backend interacts with the database and forensic engine. The watchdog module ensures continuous monitoring, enabling real-time detection and response. This integration ensures smooth data flow, coordinated processing, and reliable system performance.

233.8 Frontend Implementation : The frontend is developed using HTML, CSS, and JavaScript, providing a user-friendly interface for interacting with the system. It supports functionalities such as file upload, forensic data visualization, and file management. The system uses Chart.js to display graphical representations including file status distribution, risk level analysis, and scan history trends. Real-time updates are handled using JavaScript to ensure dynamic interaction with the backend.

3.4 Testing

Testing is performed to ensure that the system functions correctly, securely, and efficiently under different conditions. Multiple levels of testing are conducted to validate individual components as well as the complete system workflow.

3.4.1 Unit Testing

Unit testing verifies individual modules independently to ensure that each component performs its intended function correctly.

Test ID	Module	Test Scenario	Expected Result	Status
UT-01	File Upload	Upload valid file	File stored successfully	Pass
UT-02	File Upload	Upload invalid file	Error message	Pass
UT-03	Hashing	Generate SHA256	Correct hash generated	Pass
UT-04	Scanner	Detect modified file	Status updated	Pass
UT-05	Database	Insert record	Record stored	Pass
UT-06	CSV Export	Export report	CSV generated	Pass

3.4.2 Integration Testing

Integration testing verifies communication between different system modules such as frontend, backend, database, and scanner.

Test ID	Modules	Scenario	Expected Result	Status
IT-01	Frontend + Backend	Upload file	File processed	Pass
IT-02	Backend + DB	Store data	Data saved	Pass
IT-03	Backend + Scanner	Trigger scan	Scan executed	Pass
IT-04	Scanner + DB	Save results	Data stored	Pass
IT-05	Backend + CSV	Export report	CSV downloaded	Pass

3.4.3 System Testing

System testing evaluates the complete system to ensure all components work together correctly.

Test ID	Scenario	Expected Result	Status
ST-01	Upload → Scan	Status displayed	Pass
ST-02	Modify file	Detected correctly	Pass
ST-03	Delete file	Marked deleted	Pass
ST-04	Dashboard load	Data visible	Pass
ST-05	CSV export	File downloaded	Pass

3.4.4 Security Testing

Security testing ensures the system is protected against common vulnerabilities and unauthorized access.

Test ID	Scenario	Expected Result	Status
SEC-01	SQL Injection	Blocked	Pass
SEC-02	Invalid file upload	Rejected	Pass
SEC-03	Path traversal	Prevented	Pass
SEC-04	Unauthorized access	Denied	Pass

3.4.5 Performance Testing

Performance testing evaluates system behavior under load and concurrent usage conditions.

Test ID	Scenario	Expected Result	Status
PT-01	Multiple file uploads	No crash	Pass
PT-02	Large file scan	Completed	Pass
PT-03	Concurrent access	Stable	Pass

3.5 Graphical User Interface (GUI) Layout

The Graphical User Interface (GUI) of the system is designed to be simple, intuitive, and efficient for both general users and administrators. The interface ensures smooth navigation, clear visualization of forensic data, and easy interaction with system functionalities.

The design follows user-centered principles to provide a seamless experience. It minimizes complexity while maintaining functionality, allowing users to perform operations without requiring technical expertise.

The GUI is divided into two main sections:

- Client Interface
- Admin Dashboard

3.5.1 Client Interface

- The client interface serves as the primary interaction point for users. It enables users to perform essential file-related operations such as uploading, viewing, editing, and downloading files.

The interface is designed using HTML and CSS to provide a clean and structured layout, while JavaScript enables dynamic interaction and real-time updates.

Key Features

- **File Upload Functionality:** Users can easily select and upload files for forensic analysis. The upload process is secure and efficient.
- **File List View:** Displays all uploaded files in a structured format, allowing users to access and manage files easily.
- **File Viewing Option:** Users can view file contents directly within the interface, enabling quick inspection of data.
- **File Editing Capability:** Allows modification of file content and saving of changes, supporting dynamic updates.
- **Download Functionality:** Users can download files for external use or analysis.
- **Navigation Controls:** Provides easy navigation between file operations and the dashboard.

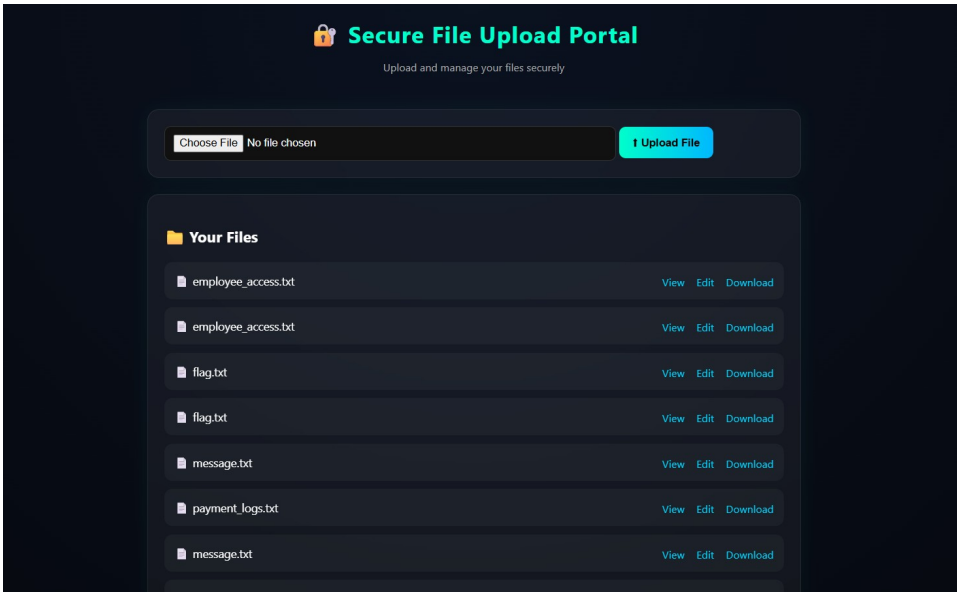


Figure 2.5.1: Client Interface – Filw upload Page and File List View

3.5.2 Admin Dashboard

The admin dashboard provides a centralized interface for monitoring and analyzing forensic data. It presents system information in a structured and visually appealing manner.

The dashboard helps administrators quickly identify suspicious activities and analyze file changes using graphical representations.

Key Features

- **File Status Overview;** Displays statistics such as total files, modified files, new files, and unchanged files.
- **Risk Level Indicators:** Uses color-coded indicators (LOW, MEDIUM, HIGH, CRITICAL) to highlight potential threats.
- **Graphical Visualization:** Provides charts and graphs for better understanding of system activity.
- **Scan History Tracking:** Displays trends in file changes over time.
- **Evidence Log Table:**Shows detailed forensic data including file status, hash values, and timestamps.
- **CSV Export Option:** Allows exporting forensic data for reporting and external analysis.

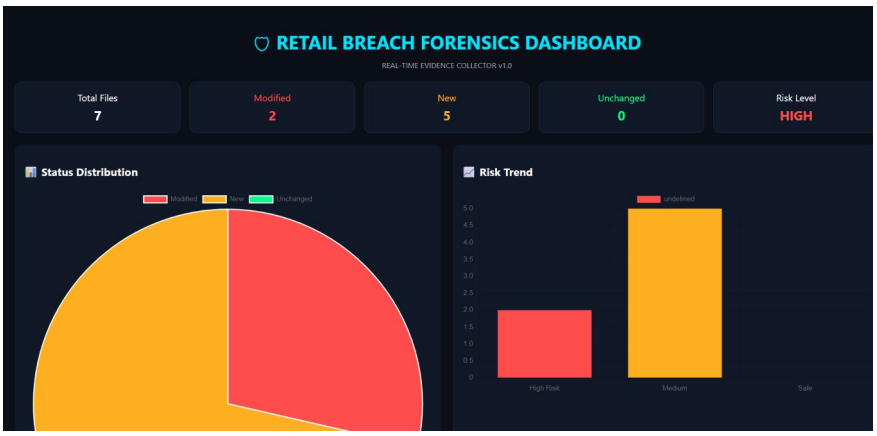


Figure 2.5.2 : Admin Dashboard Overview and Risk Level Trend

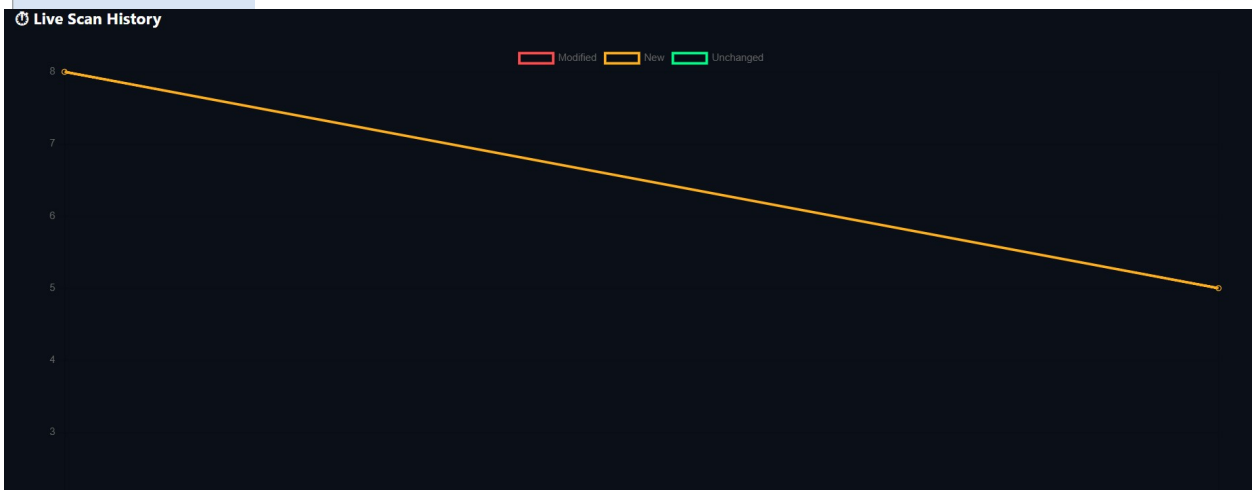


Figure 2.5.2 : Scan History Graph

This figure shows the live scan history of the system, representing file activity over time. The graph illustrates changes in file status such as new, modified, and unchanged files, helping users track trends and detect unusual patterns. The visualization provides a time-based analysis of file operations, enabling administrators to monitor system behavior and identify potential security issues. It enhances understanding of how file activity evolves and supports effective forensic investigation.

3.5.3 GUI Design Principles

The overall GUI design follows standard usability principles:

- **Consistency** – Uniform design across all pages
- **Feedback** – System provides immediate response to user actions
- **Error Prevention** – Input validation and clear warnings
- **Efficiency** – Reduces number of steps for operations
- **Aesthetic Design** – Clean and modern interface

3.5.4 Summary

The GUI layout ensures that users can efficiently interact with the system while maintaining clarity and usability. The client interface supports file operations, while the admin dashboard

provides analytical insights through visualizations and reports. Together, they create a complete and user-friendly system for foren

3.6 Customer Testing

Customer testing is conducted to evaluate the system from the end-user perspective and ensure that it meets user requirements in terms of usability, performance, and functionality.

Testing Process

Users were asked to upload files and perform operations such as viewing, editing, and deleting files. The system automatically processed the files and displayed results on the dashboard. User interactions and system responses were carefully observed.

Observations

- The system was easy to use with a simple and intuitive interface
- File upload and scanning operations were fast and efficient
- The dashboard provided clear insights into file status and risk levels
- The CSV export feature was useful for reporting and analysis

Conclusion

Customer testing confirmed that the system performs effectively in real-world scenarios and meets user expectations. The system provides a reliable, user-friendly, and efficient solution for forensic analysis.

3.7 Evaluation

The system is evaluated to assess its overall performance, usability, security, and reliability in achieving the project objectives. The evaluation focuses on how effectively the system performs real-time monitoring, detects file changes, and provides meaningful forensic insights through a user-friendly interface.

Project Achievements

The project successfully implements a complete file monitoring and forensic analysis workflow. The system enables real-time detection of file changes using the watchdog module, ensuring that any modification, deletion, or addition is immediately identified.The use of SHA256 hashing provides accurate integrity verification, allowing even minor changes in files to be detected reliably. The frontend and backend are effectively integrated using REST APIs, enabling smooth communication between system components.Additionally, the dashboard presents clear and structured visualization of file status and risk levels, helping users quickly analyze system activity.

Strengths

The system has a modular and scalable architecture, allowing different components such as file handling, scanning, and monitoring to work independently while remaining well integrated.Real-time monitoring and automatic scanning improve efficiency by eliminating the need for manual checks. The hashing mechanism ensures accurate and reliable detection of file changes.The user interface is simple and intuitive, making the system easy to use for both technical and non-technical users. The use of PostgreSQL ensures efficient and secure storage of forensic data.

Limitations

The system is designed for small to medium-scale environments and may require optimization for handling very large datasets. It does not include advanced machine learning techniques for anomaly detection, which could further enhance threat detection.Authentication and access control mechanisms are basic and can be improved for higher security requirements. Additionally, testing is primarily manual, with limited automation.

Performance Evaluation

The system performs efficiently under normal conditions. File upload, scanning, and data processing operations are completed quickly, and real-time monitoring ensures immediate detection of file changes.The database handles storage and retrieval operations effectively, maintaining smooth system performance.

Usability Evaluation

The interface is user-friendly and easy to navigate. Users can perform file operations and access dashboard insights without difficulty. The use of charts and structured data presentation improves clarity and helps users quickly understand system activity.

Security Evaluation

The system ensures data integrity using SHA256 hashing and implements secure file handling practices. Input validation and parameterized database queries help protect against common vulnerabilities such as invalid inputs and SQL injection.

Reliability Evaluation

The system consistently detects file changes and maintains accurate records. The watchdog module ensures continuous monitoring, while the database securely stores all forensic evidence.

Comparative Analysis

The proposed system offers improvements over traditional manual monitoring methods.

Feature	Traditional System	Proposed System
Monitoring	Manual	Automated
Detection Speed	Slow	Real-time
Accuracy	Moderate	High
Reporting	Manual	Automated
Visualization	Limited	Interactive

Overall Evaluation

Overall, the system successfully achieves its objectives by providing an efficient, reliable, and user-friendly solution for file monitoring and forensic analysis. It demonstrates strong performance and accurate detection capabilities, while also offering clear visualization for better decision-making

Evaluation Results

Parameter	Result
File Upload Time	Fast
Scan Processing Time	< 2 seconds
Change Detection Accuracy	High
System Stability	Stable
Data Integrity	Maintained

Conclusion

Overall, the system successfully meets its objectives by providing an efficient, secure, and reliable solution for file monitoring and forensic analysis. It demonstrates strong performance and usability, making it suitable for real-world applications with scope for future enhancements.

4. Snapshots of the Project

This section includes screenshots of the system demonstrating its functionality and user interface.

- **Client Interface**

Figure 3.1: File Upload Page

This figure shows the file upload interface where users can select and upload files for forensic analysis. It provides a simple and efficient way to submit files into the system.

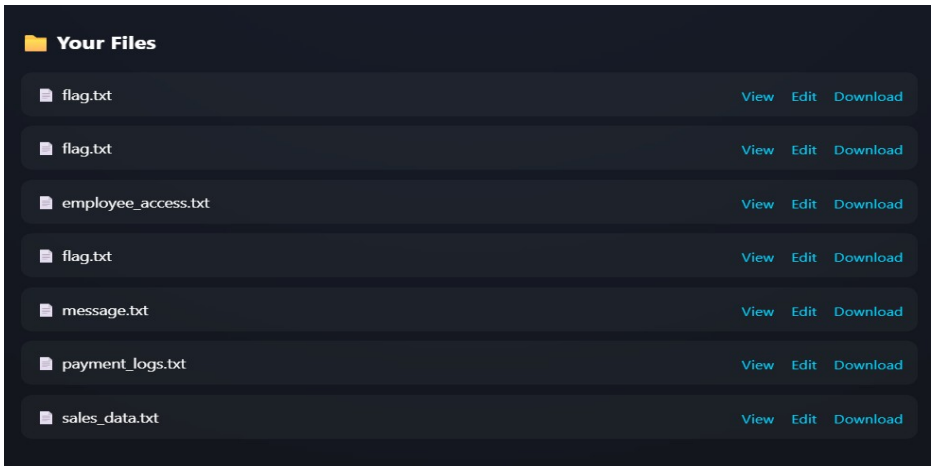


Figure 3.2: File List Page

This figure displays the list of uploaded files along with available actions such as view, edit, and download. It enables efficient file management within the system.

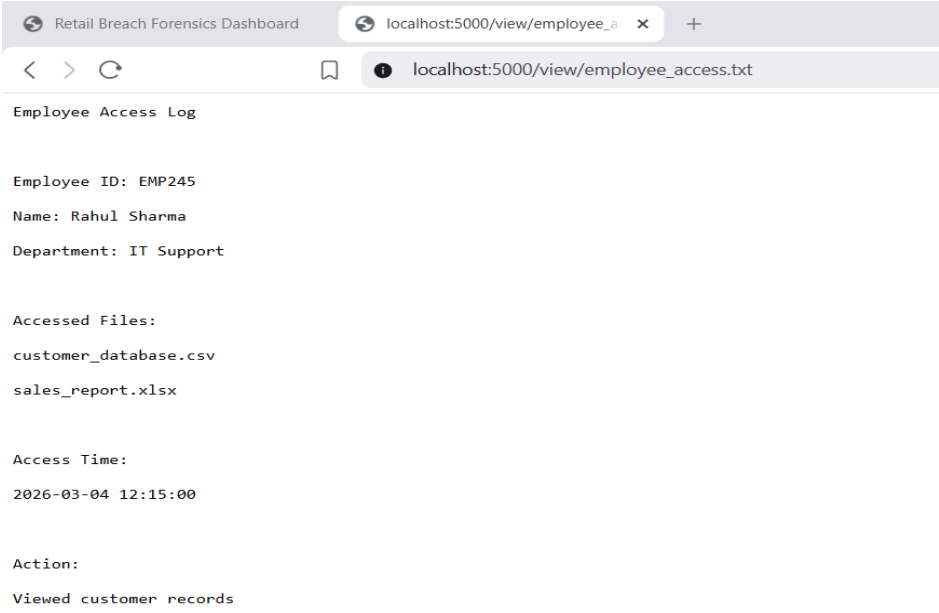


Figure 3.3 File View Page

This figure shows the file viewing interface where users can view the contents of selected files. It presents file data in a readable format for analysis.

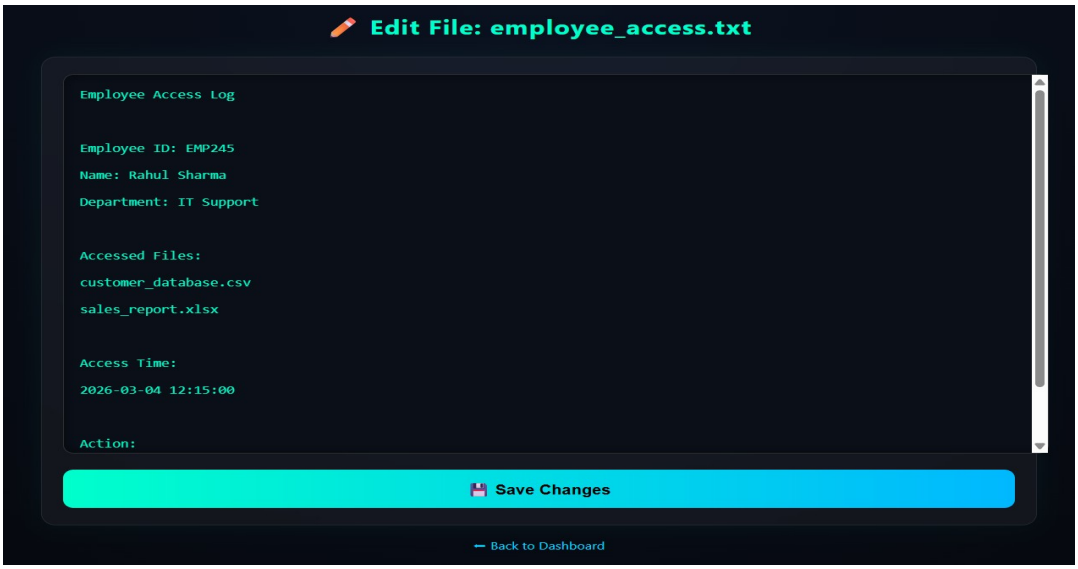


Figure 3.4: File Edit Page

This above figure shows the file editing interface that allows users to modify file content and save changes. It supports dynamic updates to files within the system.

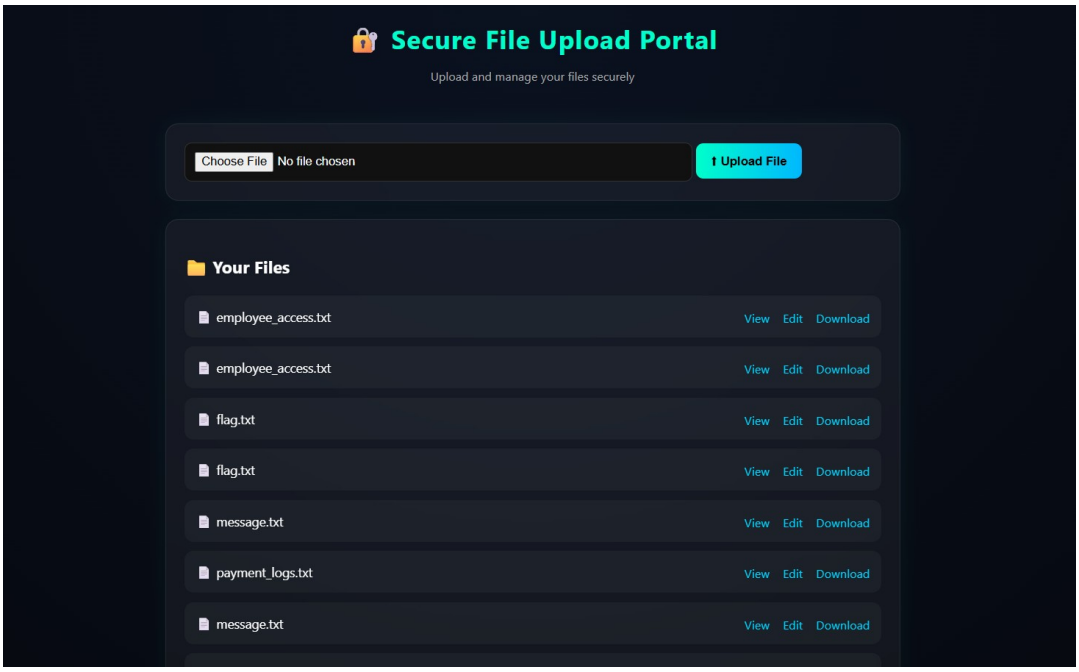


Figure 3.5 Complete Client Interface

The client interface is designed to provide a simple and user-friendly environment for interacting with the system. It allows users to perform essential file operations such as uploading, viewing, editing, and downloading files.The interface includes a file upload section where users can select files and submit them for forensic analysis. Once uploaded, files are displayed in a structured list format, providing options to view, edit, or download each file.

The design ensures easy navigation and efficient interaction, enabling users to manage files without requiring technical expertise. Dynamic updates using JavaScript allow real-time interaction with the backend system, ensuring a smooth user experience.Overall, the client interface provides an intuitive platform for file management and serves as the entry point for forensic processing within the system.

► Admin Dashboard

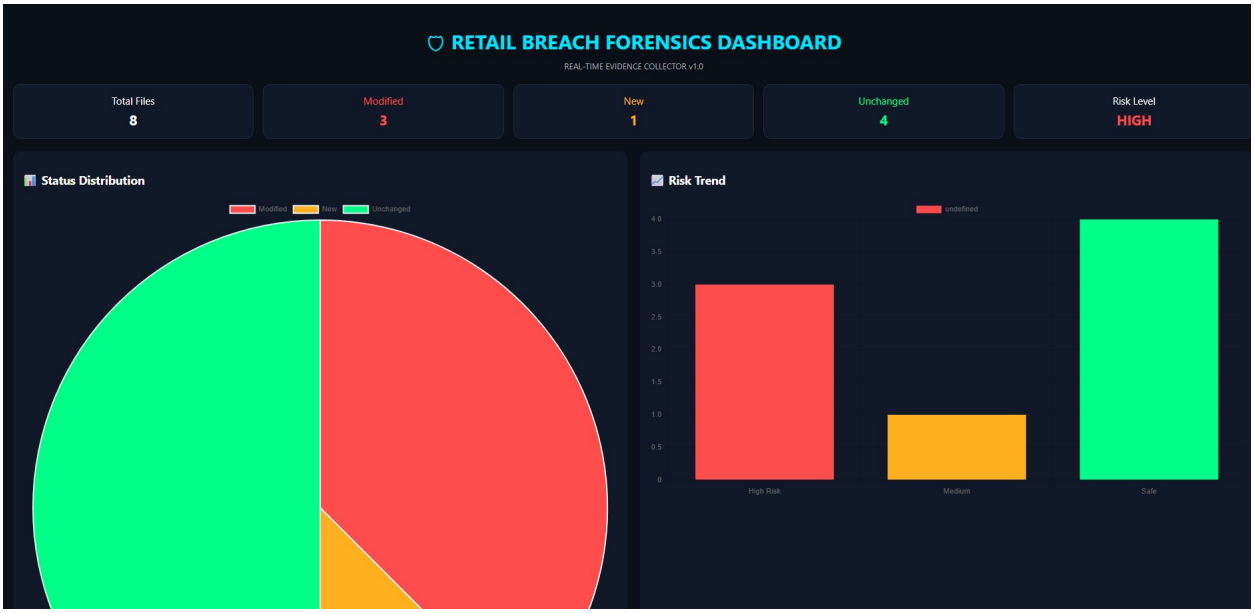


Figure 3.6: Admin Dashboard Overview

This figure displays the dashboard overview showing key metrics such as total files, modified files, new files, unchanged files, and overall risk level. It provides a quick summary of system activity.

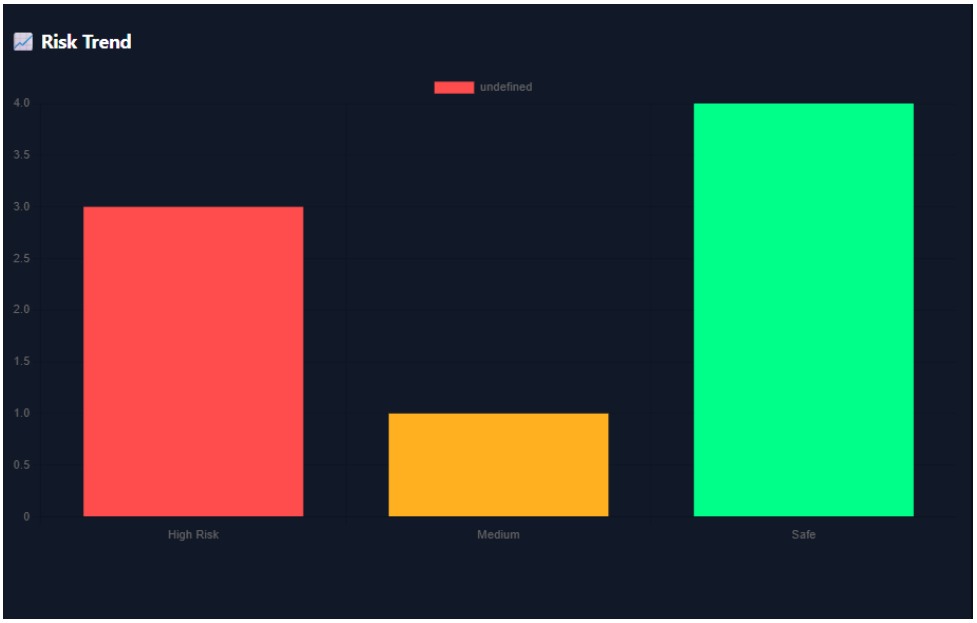


Figure 3.7 Risk Trend Chart

This above figure represents the risk level distribution of files using graphical visualization. It helps users identify high-risk and low-risk files easily.

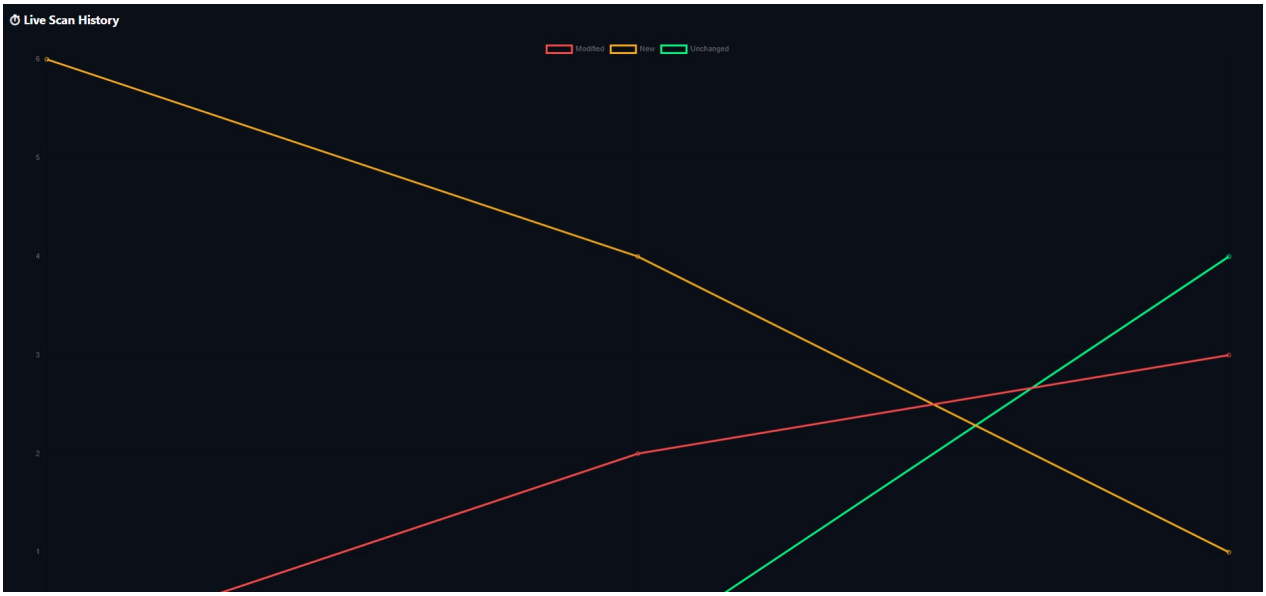
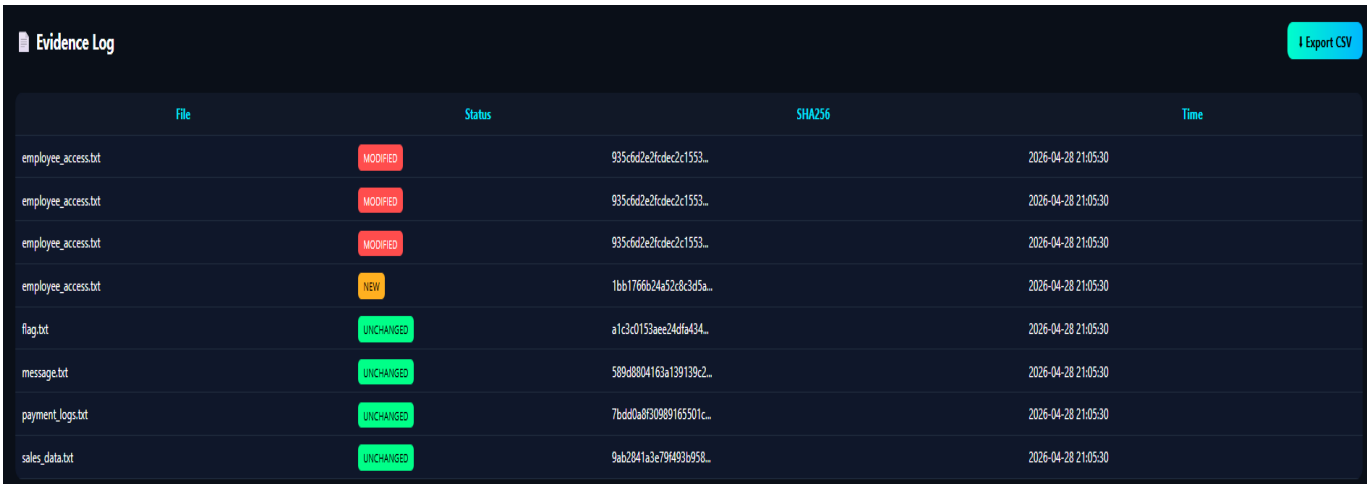


Figure 3.8 Live Scan History

This above figure shows the scan history of files over time. It helps track changes and analyze trends in file activity.



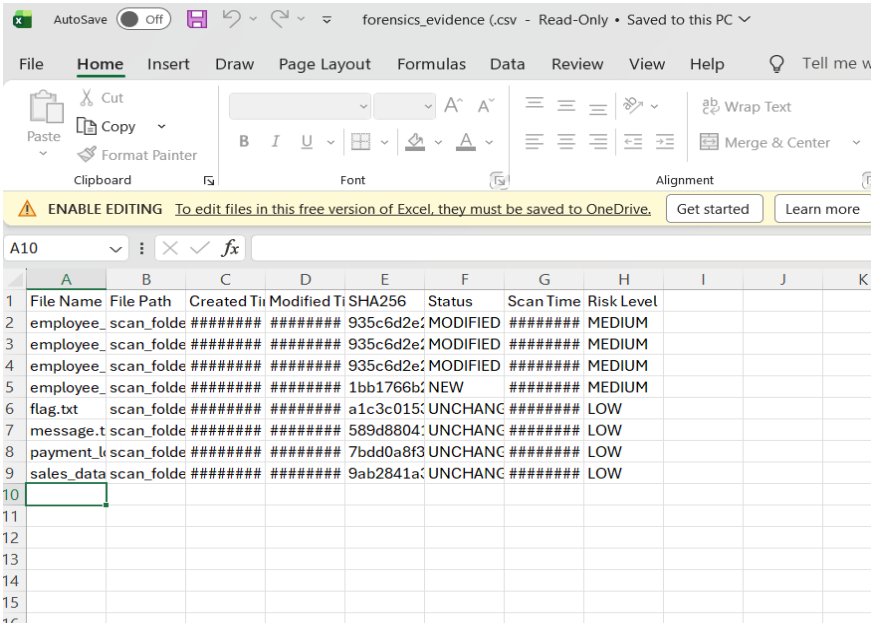
Evidence Log

Export CSV

File	Status	SHA256	Time
employee_access.txt	MODIFIED	935c6d2e2fcdec2c1553...	2026-04-28 21:05:30
employee_access.txt	MODIFIED	935c6d2e2fcdec2c1553...	2026-04-28 21:05:30
employee_access.txt	MODIFIED	935c6d2e2fcdec2c1553...	2026-04-28 21:05:30
employee_access.txt	NEW	1bb17766b24a52c8c3d5a...	2026-04-28 21:05:30
flag.txt	UNCHANGED	a1c3c0153aee24dfa434...	2026-04-28 21:05:30
message.txt	UNCHANGED	589d8804163a139139c2...	2026-04-28 21:05:30
payment_logs.txt	UNCHANGED	7bdd0a8f30989165501c...	2026-04-28 21:05:30
sales_data.txt	UNCHANGED	9ab2841a3e79f493b958...	2026-04-28 21:05:30

Figure 3.9 : Evidence log (Scan Results Table) And Export CSV Report Button

This above figure displays the scan results table containing file details such as file name, status, SHA256 hash, and timestamp. It provides structured forensic evidence

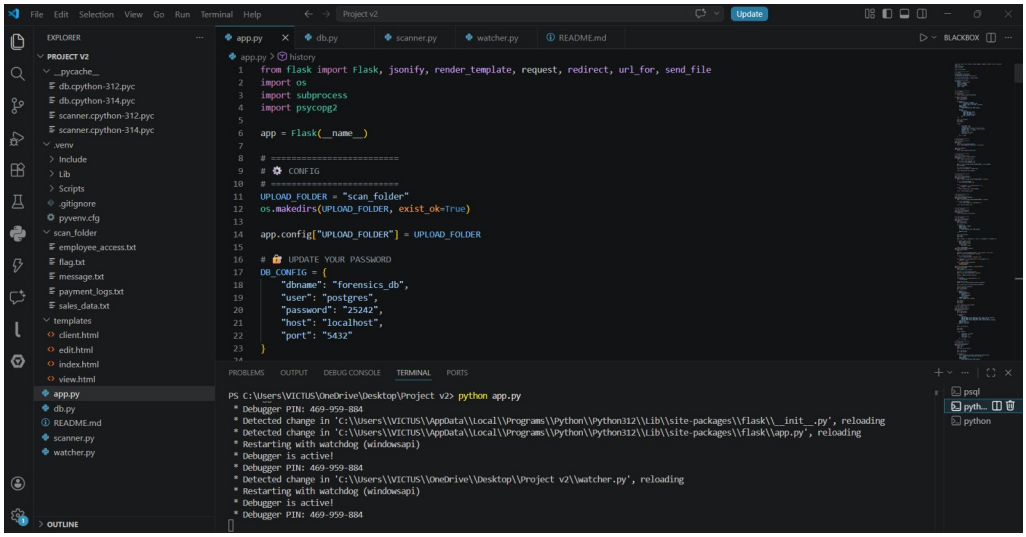


File Name	File Path	Created Ti	Modified Ti	SHA256	Status	Scan Time	Risk Level
employee_	scan_folde	########	########	935c6d2e2	MODIFIED	########	MEDIUM
employee_	scan_folde	########	########	935c6d2e2	MODIFIED	########	MEDIUM
employee_	scan_folde	########	########	935c6d2e2	MODIFIED	########	MEDIUM
employee_	scan_folde	########	########	1bb1766b2	NEW	########	MEDIUM
flag.txt	scan_folde	########	########	a1c3c0153	UNCHANG	########	LOW
message.t	scan_folde	########	########	589d8804	UNCHANG	########	LOW
payment_l	scan_folde	########	########	7bdd0a8f3	UNCHANG	########	LOW
sales_data	scan_folde	########	########	9ab2841a3	UNCHANG	########	LOW

Figure 3.9 : CSV Export Output

This above figure shows the exported CSV report containing forensic data. It allows users to analyze, store, and share results externally.

- **Project execution**



```
from flask import Flask, jsonify, render_template, request, redirect, url_for, send_file
import os
import subprocess
import psycopg2

app = Flask(__name__)

# ========================
# ⚙️ CONFIG
# ========================
UPLOAD_FOLDER = "scan_folder"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

# 🔐 UPDATE YOUR PASSWORD
DB_CONFIG = {
    "dbname": "forensics_db",
    "user": "postgres",
    "password": "25242",
    "host": "localhost",
    "port": "5432"
}
```

```
PS C:\Users\VICTUS\OneDrive\Desktop\Project v2> python app.py
 * Debugger PIN: 469-959-884
 * Detected change in 'C:\\Users\\VICTUS\\AppData\\Local\\Programs\\Python\\Python312\\Lib\\site-packages\\flask\\__init__.py', reloading
 * Detected change in 'C:\\Users\\VICTUS\\AppData\\Local\\Programs\\Python\\Python312\\Lib\\site-packages\\flask\\app.py', reloading
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 469-959-884
 * Detected change in 'C:\\Users\\VICTUS\\OneDrive\\Desktop\\Project v2\\watcher.py', reloading
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 469-959-884
```

Figure 3.10 : Project execution environment

This figure shows the execution of the project in the development environment using Visual Studio Code. The project structure is clearly organized into multiple modules, including *app.py* for handling the Flask backend, *scanner.py* for performing forensic analysis and hash generation, *watcher.py* for real-time file monitoring, and *db.py* for managing database operations.

The terminal window displays the Flask application running in debug mode, indicating that the server is active and ready to process incoming requests. It also shows automatic reloading functionality, which updates the application whenever changes are made to the source code during development.

This setup allows efficient testing and debugging of the system, ensuring that all components such as file monitoring, scanning, and database interactions work correctly. Overall, the figure demonstrates the successful execution and integration of the system within the development environment.

► **Folder Structure**

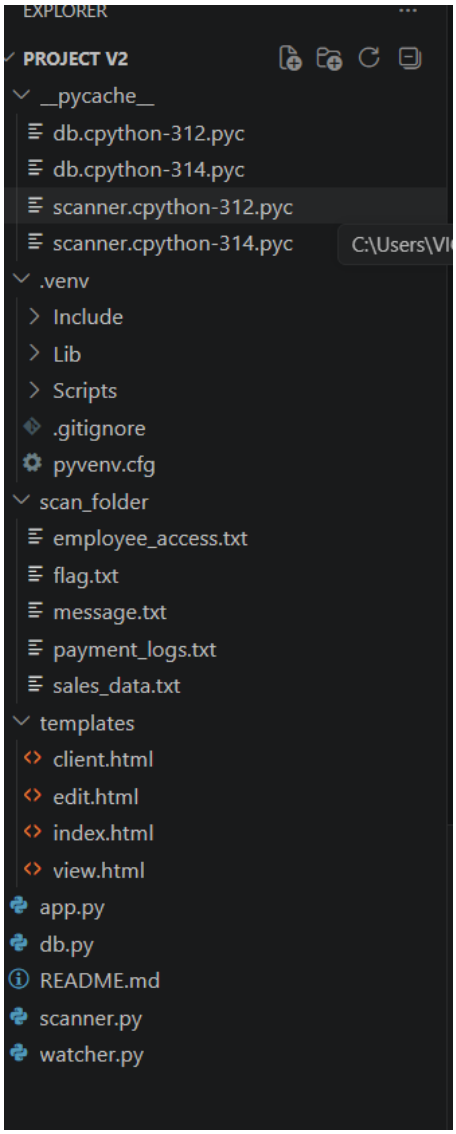


Figure 3.9 : Folder Structure of Project

This above figure shows the overall folder structure of the project as organized in the development environment. The project follows a modular structure to ensure maintainability and clarity.

The main application logic is implemented in app.py, which handles routing, API requests, and integration between components. The scanner.py module is responsible for performing forensic analysis and generating file hashes, while watcher.py enables real-time monitoring of file changes using the watchdog library. The db.py file manages database operations and interactions with the PostgreSQL database.

The templates folder contains HTML files such as client.html, index.html, edit.html, and view.html, which define the frontend user interface. The scan_folder directory stores all uploaded and monitored files used for forensic analysis.

Additional folders such as .venv represent the virtual environment used for dependency management, and pycache stores compiled Python files for performance optimization.

This structured organization improves code readability, supports modular development, and simplifies future enhancements.

5. CONCLUSION

The Cloud Forensics Automation for Retail Breaches system presents a practical, scalable, and efficient solution for monitoring and analyzing file-level changes in a digital retail environment. By combining real-time monitoring, cryptographic hashing, structured data storage, and visual analytics, the system provides meaningful insights into file activity and potential security threats.

The project demonstrates how automation can significantly improve the speed, accuracy, and reliability of forensic analysis compared to traditional manual approaches. Through continuous monitoring and intelligent classification of file changes, the system enables rapid detection of unauthorized modifications and enhances overall system security.

5.1 Key Outcomes of the Project

The project successfully meets its primary objectives and delivers a complete end-to-end forensic monitoring solution. Key outcomes include:

- Development of an automated file monitoring system
- Implementation of SHA256-based integrity verification
- Classification of files based on status and risk level
- Secure storage of forensic evidence using PostgreSQL
- Interactive dashboard for visualization and analysis
- CSV export functionality for reporting
- Modular and scalable system architecture

5.2 Technical Contributions

The system contributes significantly to the field of digital forensics and system monitoring by:

- Enhancing data integrity verification using cryptographic hashing
- Providing real-time monitoring through automated detection mechanisms
- Supporting structured forensic analysis using database storage
- Improving system efficiency through automation of monitoring tasks

5.3 Real-World Applications

In modern retail environments, data security is critical due to increasing cyber threats and reliance on digital systems.

The system can be applied in:

- Retail data protection systems
- File integrity monitoring in organizations
- Security auditing and compliance monitoring
- Digital forensic investigations

It helps reduce risks such as data breaches, unauthorized modifications, and system misuse.

5.4 Key Strengths

1. Real-Time Monitoring
    - Continuous tracking of file changes
2. Accurate Detection Mechanism
    - SHA256 ensures precise change detection
3. User-Friendly Interface
    - Simple and intuitive dashboard
4. Efficient Data Management
    - Structured storage using PostgreSQL
5. Modular Architecture
    - Easy to maintain and extend

5.5 System Limitations

1. Limited scalability for very large datasets
2. No AI-based anomaly detection
3. Basic security features (limited authentication)
4. Requires manual configuration of monitoring directory
5. Limited cloud integration

5.6 Final Remarks

The Cloud Forensics Automation for Retail Breaches system successfully demonstrates an effective approach to automated forensic monitoring. It integrates real-time tracking, cryptographic validation, and data visualization to provide a comprehensive solution for detecting file-level threats.

The project not only achieves its objectives but also highlights the importance of automation in improving security and efficiency in modern digital systems. With further enhancements, the system has strong potential for real-world deployment in enterprise environments.

6. FURTHER DEVELOPMENT OR RESEARCH

The proposed system provides a strong foundation for automated forensic monitoring; however, there are several opportunities for further enhancement and research. These improvements can significantly increase the system's scalability, intelligence, and applicability in real-world enterprise environments.

6.1 Future Enhancements

1. Cloud Integration

The system can be extended to support cloud-based storage platforms such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud. This would enable:

- Monitoring of files stored in cloud environments
- Improved scalability and remote accessibility
- Better data backup and disaster recovery

Cloud integration would allow the system to function effectively in modern distributed infrastructures.

2. Machine Learning-Based Anomaly Detection

Currently, the system relies on rule-based detection using hashing. Future work can incorporate machine learning techniques to:

- Identify unusual patterns in file behavior
- Detect advanced threats and anomalies
- Improve accuracy in identifying suspicious activities

This would enhance the intelligence of the system beyond basic change detection.

3. User Authentication and Access Control

The system can be enhanced by implementing authentication and role-based access control mechanisms. This would:

- Restrict access to authorized users
- Provide different access levels (admin, user, analyst)
- Improve overall system security

Technologies such as JWT or OAuth can be used for secure authentication.

4. Real-Time Alert System

Future versions can include real-time notification systems such as:

- Email alerts
- SMS notifications
- Push notifications

This would allow users to receive immediate alerts when critical file changes are detected, improving response time to potential threats.

5. Support for Large-Scale Distributed Systems

The current system is suitable for small to medium-scale environments. Future enhancements can focus on:

- Handling large volumes of files
- Distributed monitoring across multiple systems
- Load balancing and performance optimization

This would make the system suitable for enterprise-level deployment.

6. Mobile Application Development

A mobile application can be developed to provide remote access to system data. This would allow users to:

- Monitor file activity on the go
- Receive real-time alerts
- Access reports and dashboards

This improves accessibility and user convenience.

6.2 Research Opportunities

The project also opens up several research directions:

- Integration of AI for predictive threat analysis
- Blockchain-based secure evidence storage
- Advanced forensic analytics and visualization
- Automated incident response systems

These areas can further enhance the capabilities of digital forensic systems.

6.3 Summary

The proposed enhancements and research directions demonstrate that the system has strong potential for future growth. By incorporating advanced technologies and expanding its capabilities, the system can evolve into a comprehensive and enterprise-ready forensic monitoring solution.

7. REFERENCES

This section lists all academic resources, technical documentation, standards, and online materials referenced during the development of the project. The references are categorized for clarity and are aligned with professional documentation practices. All sources are relevant to

digital forensics, file monitoring, security, and web-based system development.

7.1 Academic and Research Publications

These references provide foundational knowledge in digital forensics, file integrity verification, and security analysis.

BrianCarrier : File System Forensic Analysis A comprehensive resource on forensic techniques for analyzing file systems and detecting digital evidence.

Eoghan Casey : Digital Evidence and Computer Crime Provides in-depth knowledge of digital forensic investigation methods and handling of electronic evidence.

Simson Garfinkel : Digital Forensics Research Discusses modern challenges and advancements in digital forensic analysis and automated investigation systems.

7.2 Industry Standards and Security Guidelines

These standards influence system design, security considerations, and forensic practices.

OWASP Top 10 :Web Application Security Risks Industry-standard guidelines for identifying and mitigating common security vulnerabilities.

NIST Guidelines on Computer Security and Monitoring Provides best practices for secure system monitoring, data protection, and forensic processes.

ISO/IEC 27001: Information Security Management Systems Defines standards for maintaining data security, integrity, and system reliability.

7.3 Technical Documentation and Tools

These resources were used for system development, implementation, and integration.

Python Documentation: Official documentation for Python programming used in backend development.

Flask Documentation : Provides guidelines for building web applications and REST APIs.

PostgreSQLDocumentation : Used for database design, queries, and data management.

Watchdog Library Documentation : Used for implementing real-time file monitoring.

Chart.js Documentation: Used for creating visualizations such as charts and graphs in the dashboard.

8. APPENDIX

The appendix provides additional supporting materials that complement the main content of the project. It includes technical details, sample data, and supplementary information used during the development and implementation of the system.

8.1 Sample Database Schema

The system uses a PostgreSQL database to store forensic evidence. The main table structure is shown below:

Table: Evidence

Field Name	Data Type	Description
file_name	VARCHAR	Name of the file
file_path	TEXT	Location of the file
sha256	TEXT	Hash value of the file
status	VARCHAR	File status (NEW, MODIFIED, etc.)
risk_level	VARCHAR	Risk classification
scan_time	TIMESTAMP	Time of scan

This schema ensures efficient storage and retrieval of forensic data.

8.2 API Endpoint List

The system provides RESTful APIs for communication between frontend and backend.

Endpoint	Method	Description
/api/evidence	GET	Fetch all scan results
/api/summary	GET	Retrieve dashboard statistics
/api/history	GET	Get scan history data
/api/export	GET	Export data as CSV
/api/scan	POST	Trigger manual scan

These APIs enable seamless integration and real-time data interaction.

8.3 Sample CSV Output

The system allows exporting forensic data in CSV format. A sample structure is shown below:

file_name	status	sha256	risk_level	scan_time
file1.txt	MODIFIED	abc123...	HIGH	2026-04-28 10:30
file2.txt	NEW	xyz456...	MEDIUM	2026-04-28 10:32

This format supports easy analysis using spreadsheet tools.

8.4 System Workflow Steps

The system follows a structured workflow for processing files:

1. User uploads a file through the client interface
2. File is stored in the scan_folder directory
3. Watchdog monitors file system events
4. Scanner generates SHA256 hash
5. Hash is compared with previous records
6. File status is determined (NEW, MODIFIED, etc.)
7. Data is stored in PostgreSQL
8. Results are displayed on the dashboard

This workflow ensures smooth and automated forensic analysis